Creating LLM Concept Sets

Joel N. Swerdel

2026-04-09

Contents

1 Introduction **2**

2 Overview of Process **2**
2.1 Selecting the health condition and the SNOMED concept 2
2.2 Gather your database information 2
2.3 Create your LLM connection information 2

3 Output **3**
3.1 Output files 3
3.2 Output results 3

1 Introduction

The `Phenelope` package enables evaluating the creating of a health condition concept set. The output of the process is a JSON file that can be imported directly into ATLAS to a concept set.

This vignette describes how to run the Phenelope process from start to end in the `Phenelope` package.

2 Overview of Process

There is only a small amount of information needed to create a concept set:

1. The name of a health condition and a SNOMED concept Id that is close to the health condition.
2. Database information to access the database and the associated vocabulary.
3. A LLM connection object.

Each of these steps is described in detail below. For this vignette, we will describe the request for a concept set for type 2 diabetes mellitus (T2DM).

2.1 Selecting the health condition and the SNOMED concept

The health condtion name to be used for the process would be something like "Type 2 Diabetes Mellitus". After examining the database, an appropriate SNOMED concept id is 201826 for "Type 2 diabetes mellitus".

Quick Tip: Phenelope uses PHOEBE to determine possible other concepts that are not direct children of the selected concept. You could, for instance select, 201820 for "Diabetes mellitus", the parent of 201826. This will expand the number of concepts that the LLM will test. Depending on your health condition, this may or may not be desirable. The only cost to using a parent concept is the number of concepts that are eventually tested, which could increase the amount of time to create the concept set.

2.2 Gather your database information

The information you will need to gather for connecting to the database is:

1. The database connection string as is typically used in HADES packages.
2. The database schema where the common data model is located (commonly used in "cdmDatabaseSchema" in HADES packages)

Below is an example for creating a database connection string:

```
# create database connection details
connectionDetails <- DatabaseConnector::createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/ohdsi",
  user = "joe",
  password = "supersecret"
)
```

2.3 Create your LLM connection information

The information you will need is documented in the ellmer package. See, for example, the ellmer function chat_azure_openai. Required fields are LLM dependent, but, as an example, you may need: * endpoint * api_version * model * credentials

For example:

```
# create LLM client for ellmer
llmClient <- ellmer::chat_azure_openai(
  endpoint = gsub("/openai/deployments.*", "", keyring::key_get("genai_gpt4o_endpoint")),
```

```
  api_version = "2023-03-15-preview",
  model = "gpt-4o",
  credentials = function() keyring::key_get("genai_api_gpt4_key")
)
```

The example above creates an ellmer object that will connect to an Azure Open AI instance, version 4o. We recommend using a non-reasoning model for best results.

Once you have the necessary information, you are ready to create you concept set by calling the createConceptSet() function as shown below:

```
# run the
finalSet <- Penelope::createConceptSet(
  conceptName = "Type 2 Diabetes Mellitus",
  originalConceptList = c(201826),
  llmClient = llmClient,
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = "yourCDMSchema",
  outputDirectory = "c:/llmConceptSets/type2Dm"
)
```

The following are additional parameters that may be set but aren't required:

1. excludedConditions - Names of conditions to be excluded from the concept set.
2. minCount - Minimum cell subject count to use for concepts.
3. additionalInformation - Additional information for concept development. This may include any specific details that are desired for the concepts, for example, "only in women"
4. clinicalContext Optional clinical context for the LLM to determine appropriateness of a concept, for example, "following surgery" would include concepts whose name indicates it happened post-surgery.
5. belowMinimumCountApproach How to treat concepts below the minimum count. One of "TEST ALL", "TEST PHOEBE", "EXCLUDE ALL", or "INCLUDE ALL":

- "TEST ALL" = test all the concepts below the minimum count
- "TEST PHOEBE" = only test the concepts below minimum count that were recommended by PHOEBE
- "EXCLUDE ALL" = exclude from the concept set any concept below the minimum count
- "INCLUDE ALL" = include all the concepts below the minimum count

3 Output

The function returns a list with 2 elements:

Element 1: a dataframe that has all the concepts that were tested by the LLM and whether the LLM included them in the final concept set or decided they should not be included

Element 2: a json object for a concept set definition that may be imported into ATLAS

3.1 Output files

In your output folder you will have the following artifacts:

1. A JSON file for the concept set.
2. A csv file with the final results from the LLM process.

3.2 Output results

The csv file contains the following:

1. suggestedCondition - the concept that was tested

2. conceptId - the conceptId for the tested condition
3. mainCondition - the health condition
4. finalAnswer - YES, the concept should be included in the concept set or NO, it shouldn't
5. rationaleForAnswer - the reason the LLM gave for its decision to include or exclude the concept
6. confidenceLevel - the confidence level the LLM has about its decision. NOTE: this is usually only valuable for reasoning LLM models.
7. excludedConditions - the list of elements that were included in the excluded concepts (if any)
8. proposedInExcluded - whether the suggested condition was included in the excluded condtions
9. tested - whether the suggested concepts was actually tested (if the minCount and belowMinimumCountApproach options are used)
10. cost - the cost of testing the concept
11. recordCount - the record count in the PHOEBE database